REDES DE COMPUTADORES

Capacidade Máxima de Transmissão de um Chat TCP Multithread

Integrantes:
Felipe Soares Calderaro - RA: 168843;
Isabella Maria Ribeiro - RA: 168870;
Larissa Martins Sá - RA:168949.

Professor: Dr. Arlindo Conceição.

Repositório (GitHub): https://github.com/calderar0/redes-sockets-tcp

Objetivo. Medir a taxa de transmissão máxima, em Mbps, que o servidor do chat TCP consegue receber de 1 a 10 clientes que transmitem ao mesmo tempo, e identificar os limites e gargalos observados.

1. Ambiente experimental

1.1 Máquina

Foi usada uma única máquina, que executou o servidor e todos os clientes.

Item	Especificação
Equipamento	Notebook Acer Nitro ANV15-41
Processador	AMD Ryzen 7 7735HS with Radeon Graphics (Zen 3+, 6 nm): 8 núcleos / 16 threads (SMT), clock base 3,2 GHz (boost até 4,75 GHz), L2 4 MiB, L3 16 MiB
Memória RAM	16 GiB DDR5-5600 (1 módulo Apacer D23.3528W1.002, portanto canal único)
Sistema operacional	Microsoft Windows 11 Home Single Language, versão 10.0.26300, 64 bits
Compilador	GCC 16.1.0 (MinGW-w64 x86_64-msvcrt-posix-seh, build de Brecht Sanders, r4), modelo de threads POSIX (winpthreads)
Flags de compilação	-Wall -Wextra -O2 -std=gnu11 -pthread -lws2_32
API de sockets	Winsock 2.2 (o mesmo código compila com sockets POSIX em Linux)
Análise dos dados	Python 3.12.10
Energia	Na tomada (AC), plano de energia "Acer" (padrão do fabricante). O estado de energia foi registrado em cada rodada: AC em 110 de 110.
Carga de fundo	Navegador (Opera) aberto. Sem teste, a CPU do sistema medida foi ≈ 28% (média de 10 s), da qual ~1,7 núcleo era do navegador.

1.2 Configuração de rede

Item	Valor
Caminho de rede	Loopback (127.0.0.1), interface "Loopback Pseudo-Interface 1" do Windows. Sem placa de rede e sem enlace físico.
Bind do servidor	--bind 127.0.0.1 (não expõe a porta e o diálogo do firewall)
Pilha TCP	Padrões do Windows: controle de congestionamento CUBIC, ajuste automático da janela de recepção "normal", ECN desativado
Buffers de socket	Padrões do SO (SO_SNDBUF = SO_RCVBUF = 64 KiB iniciais, com ajuste automático). Algoritmo de Nagle ativo (sem TCP_NODELAY), como no chat original.

Utilizamos o loopback, que mostra a capacidade da aplicação e da pilha TCP/IP do sistema sem o limite de um enlace físico, pois rodamos apenas em uma máquina. Os valores obtidos (Gbps) ficam, portanto, acima do que uma rede (esquema de várias máquinas) real entregaria.

2. Modificações na aplicação

O chat original (uma thread por cliente, leitura de linhas e *broadcast* para os demais) foi mantido intacto como modo padrão. Foi adicionado um modo de medição ativado por --bench, e as diferenças de plataforma foram isoladas em net_compat.h.

2.1 Cliente (chat_client.c)

```
chat_client <IP> <porta> --bench <duracao_s> <bloco_bytes> [nome]
```

1. Conecta e envia o nome, como no protocolo original.
2. Espera o sinal do servidor, para que todos os clientes comecem juntos.
3. Envia o mesmo bloco de <bloco_bytes> sem parar até completar <duracao_s> segundos. Não há digitação manual. O bloco é texto com uma quebra de linha a cada 128 bytes, ou seja, o fluxo é uma sequência de mensagens válidas.
4. Fecha o sentido de escrita (shutdown(SHUT_WR)), espera o EOF do servidor (que confirma que tudo foi lido) e imprime RESULT (sent=<bytes enviados>).

2.2 Servidor (chat_server.c)

```
chat_server <porta> --bench <N> <aquecimento_s> <medicao_s> [--mode bulk|line] [--bind IPv4]
```

1. Aceita exatamente N conexões, cada uma atendida por uma thread própria.
2. Cada thread lê o nome (handshake original) e avisa a thread principal. Quando as N threads estão prontas, o servidor envia o sinal para todos.
3. Cada thread soma os bytes que recebe em um contador próprio. A thread é a única que escreve nele, e a thread principal apenas o lê, então não há mutex no caminho de recepção.
4. A thread principal descarta o aquecimento e, na janela de medição, tira duas “fotografias” dos contadores.
5. Depois da janela, continua lendo até todos os clientes encerrarem. Em seguida confere se todos ficaram ativos durante a janela inteira e imprime RESULT e CLIENT.

Modos de leitura do servidor:

- bulk (padrão): recv() com buffer de 128 KiB - Mede a capacidade máxima.
- line: lê 1 byte por recv() e monta linhas, exatamente como o read_line() do chat original, e ainda formata a mensagem "<nome>: <texto>\n" - Mede o custo do protocolo original.

3. Metodologia

3.1 Experimentos

Experimento	Clientes	Modo	Bloco	Aquecimento	Janela de medição	Repetições	Rodadas
E1 – principal	1 a 10	bulk	64 KiB	2 s	10 s	5	50
E2 – protocolo original	1 a 10	line	64 KiB	2 s	5 s	3	30
E3 – tamanho de bloco	1 e 10	bulk	1, 4, 16, 64 e 256 KiB	2 s	5 s	3	30

Total: 110 rodadas, sendo todas válidas.

Em cada rodada, os clientes transmitem por aquecimento + janela + 2 s (14 s no E1), para que todos ainda estejam transmitindo quando a janela fecha.

Foi escolhido o bloco de 64 KiB no E1 pois o E3 mostra que a taxa cresce com o bloco até 64 KiB e depois praticamente estabiliza com 10 clientes e com 64 KiB, o custo por chamada de sistema deixa de dominar e o teste mede a capacidade de transferência.

3.2 Procedimentos para garantir a comparabilidade

1. **Mesmos binários e parâmetros** (-O2, mesmo bloco, aquecimento e janela) em todas as rodadas de um experimento.
2. **Largada sincronizada -** Nenhum cliente transmite antes de todos estarem conectados (sinal do servidor: GO). Assim, a janela sempre mede N clientes simultâneos.
3. **Aquecimento descartado (2s) -** Exclui o *slow start* do TCP, o crescimento automático dos buffers, a criação de threads e o início do escalonamento.
4. **Medição feita no servidor**, sobre os bytes efetivamente entregues à aplicação, com relógio monotônico de alta resolução.
5. **Validação de cada rodada.** (a) A soma dos bytes enviados por todos os clientes deve ser igual ao total recebido pelo servidor, e foi igual em 110/110 rodadas. (b) Todo cliente deve ter recebido dados durante toda a janela e encerrado depois dela. Uma rodada que falhasse seria marcada valid=0.
6. **Ordem aleatória** (semente fixa = 42). Dentro de cada repetição, a ordem de N foi embaralhada, de modo que variações lentas (temperatura, *turbo boost*, processos de fundo) não favoreçam sistematicamente nenhum cenário.
7. **Pausa de 2s entre rodadas** e porta TCP diferente a cada rodada, para evitar interferência de conexões em TIME_WAIT.
8. **Ambiente registrado por rodada:** estado de energia (AC), CPU do sistema na janela e CPU do processo servidor.

9. **Repetições e estatística.** Para cada N são informados máximo, média, desvio-padrão, intervalo de confiança de 95% (t de Student) e coeficiente de variação.

4. Cálculo da taxa de transmissão agregada

Cada thread *i* do servidor mantém um contador bytes_i(t) com o total de bytes que seus recv() já retornaram. A thread principal lê todos os contadores em dois instantes:

- t0 = fim do aquecimento → B0 = Σ_i bytes_i(t0)
- t1 = t0 + 10s → B1 = Σ_i bytes_i(t1)

$$Throughput\ agregado\ (Mbps) = \frac{(B_1 - B_0) \times 8}{(t_1 - t_0) \times 10^6}$$

- **Bytes efetivamente recebidos:** são os bytes de *payload* TCP entregues à aplicação. Não entram cabeçalhos TCP/IP nem retransmissões. É a vazão útil vista pelo servidor.
- **Várias threads:** a soma Σ_i agrega as N threads. Como cada contador tem um único escritor é lido de forma atômica, não há perda de contagem nem disputa por *lock*.
- **Tempo:** t1 − t0 é o tempo real medido (de 10,000 a 10,114 s no E1), e não o valor nominal pedido ao sleep.
- **Prefixos:** 1 Mbps = 10^6 bit/s, como em redes.
- **Por cliente:** o mesmo cálculo com bytes_i(t1) − bytes_i(t0) dá a vazão de cada conexão, usada no índice de justiça de Jain: J = (Σx_i)² / (N·Σx_i²), onde J = 1 significa divisão perfeitamente igual.
- **CPU:** a CPU do servidor é Δ(tempo de usuário + sistema) / (t1 − t0), em núcleos. A CPU do sistema é 1 − Δocioso/Δtotal de todas as CPUs, amostrada pelo script dentro da janela.

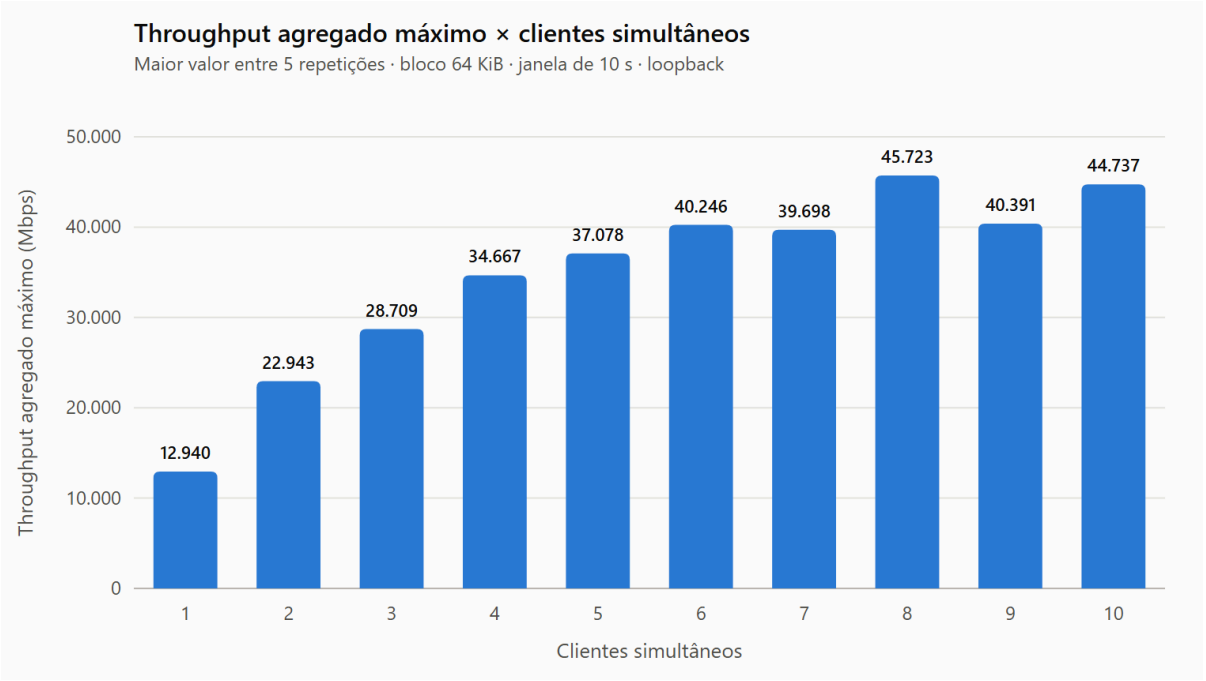
Usar uma janela interna, em vez de "bytes totais ÷ duração total", elimina o efeito do início e do fim desencontrados entre clientes e do aquecimento do TCP.

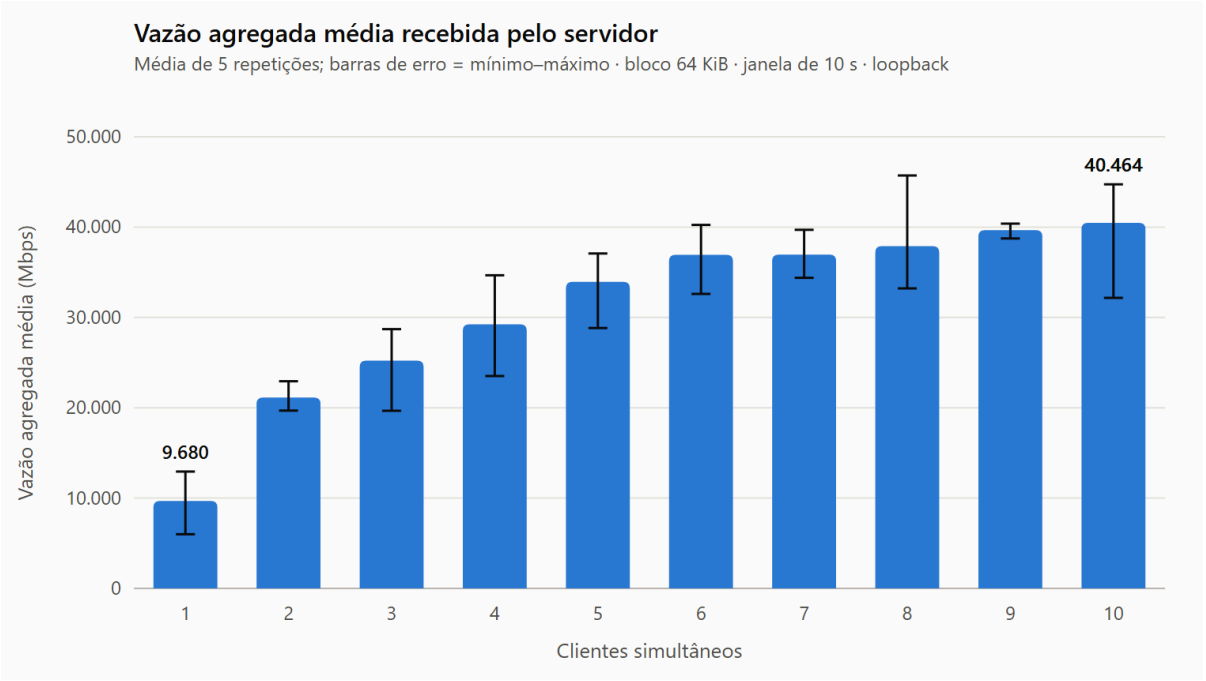
5. Resultados

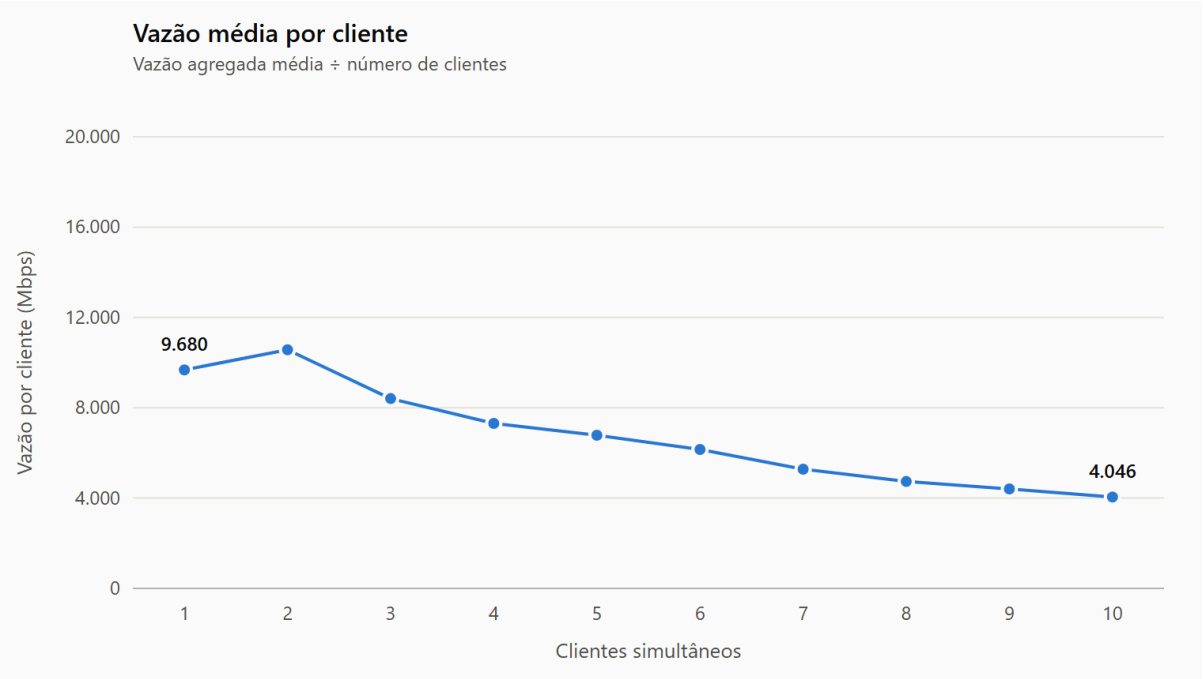
5.1 E1 – Throughput agregado × número de clientes (bulk, 64 KiB)

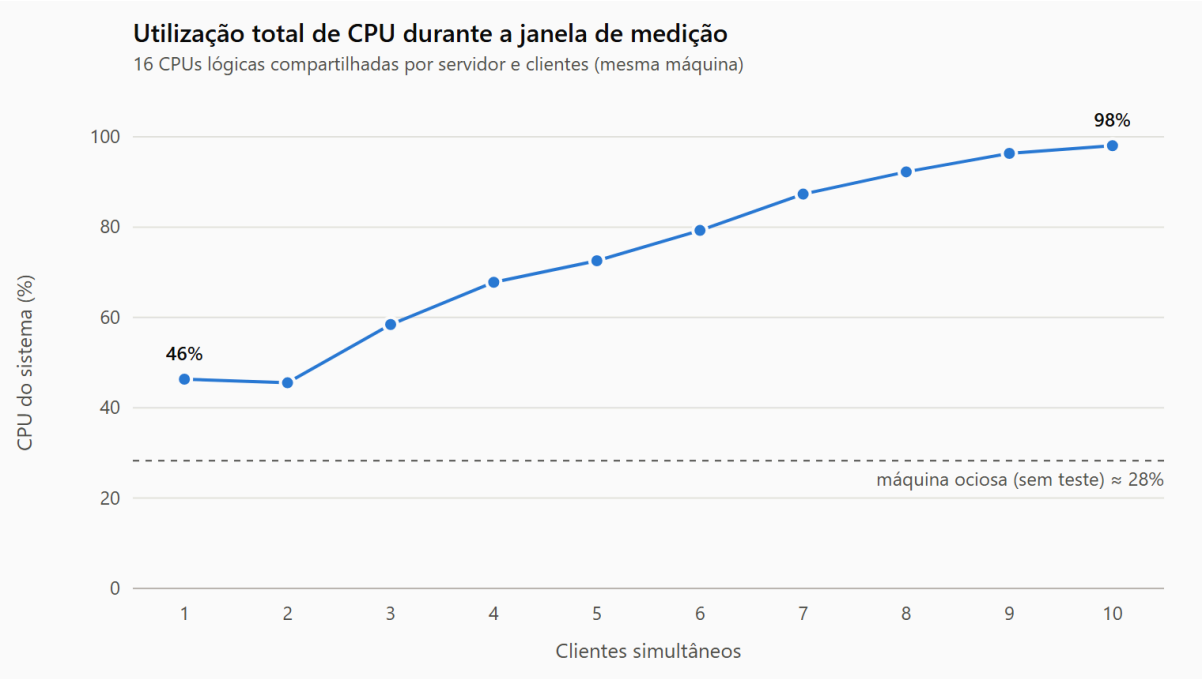
Clientes	Máximo (Mbps)	Média (Mbps)	Desvio-padrão	IC 95% (±)	CV (%)	Mínimo (Mbps)	Média por cliente (Mbps)	CPU do sistema (%)	CPU do servidor (núcleos)	Índice de Jain	Válidas
1	**12.940,3**	9.679,8	2.616,2	3.248,0	27,0	6.000,9	9.679,8	46,3	0,21	1,000	5/5
2	**22.942,6**	21.136,3	1.340,7	1.664,4	6,3	19.685,2	10.568,2	45,5	0,36	1,000	5/5
3	**28.708,7**	25.219,3	3.783,1	4.696,6	15,0	19.666,7	8.406,4	58,4	0,52	1,000	5/5
4	**34.666,5**	29.244,1	4.369,2	5.424,2	14,9	23.510,0	7.311,0	67,8	0,64	1,000	5/5
5	**37.078,0**	33.928,1	3.595,0	4.463,1	10,6	28.827,4	6.785,6	72,5	0,82	1,000	5/5
6	**40.246,5**	36.919,8	3.276,8	4.068,0	8,9	32.610,3	6.153,3	79,3	0,95	0,999	5/5
7	**39.698,1**	36.960,8	2.003,0	2.486,6	5,4	34.379,5	5.280,1	87,3	1,06	0,998	5/5
8	**45.722,6**	37.890,5	4.966,3	6.165,5	13,1	33.209,7	4.736,3	92,2	1,19	0,996	5/5
9	**40.391,5**	39.669,3	770,8	956,9	1,9	38.732,7	4.407,7	96,3	1,25	0,992	5/5

Clientes	Máximo (Mbps)	Média (Mbps)	Desvio-padrão	IC 95% (±)	CV (%)	Mínimo (Mbps)	Média por cliente (Mbps)	CPU do sistema (%)	CPU do servidor (núcleos)	Índice de Jain	Válidas
10	**44.736,7**	40.463,6	4.899,7	6.082,8	12,1	32.172,1	4.046,4	98,0	1,32	0,991	5/5





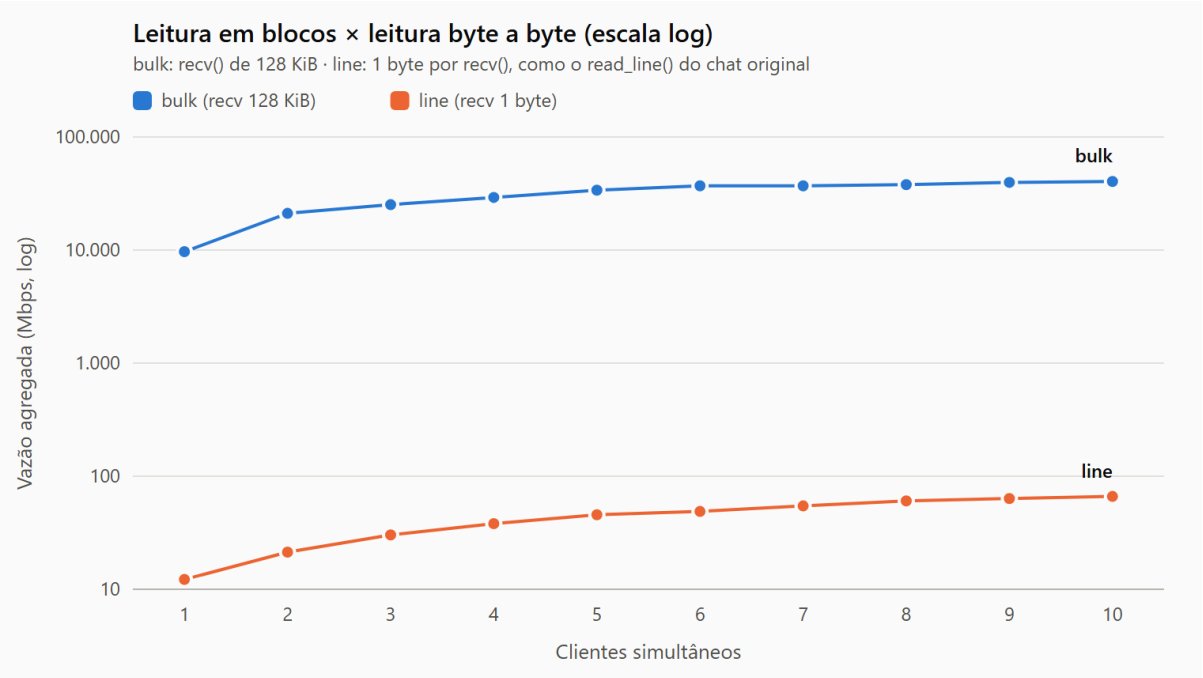




5.2 E2 – Protocolo original do chat (1 byte por recv())

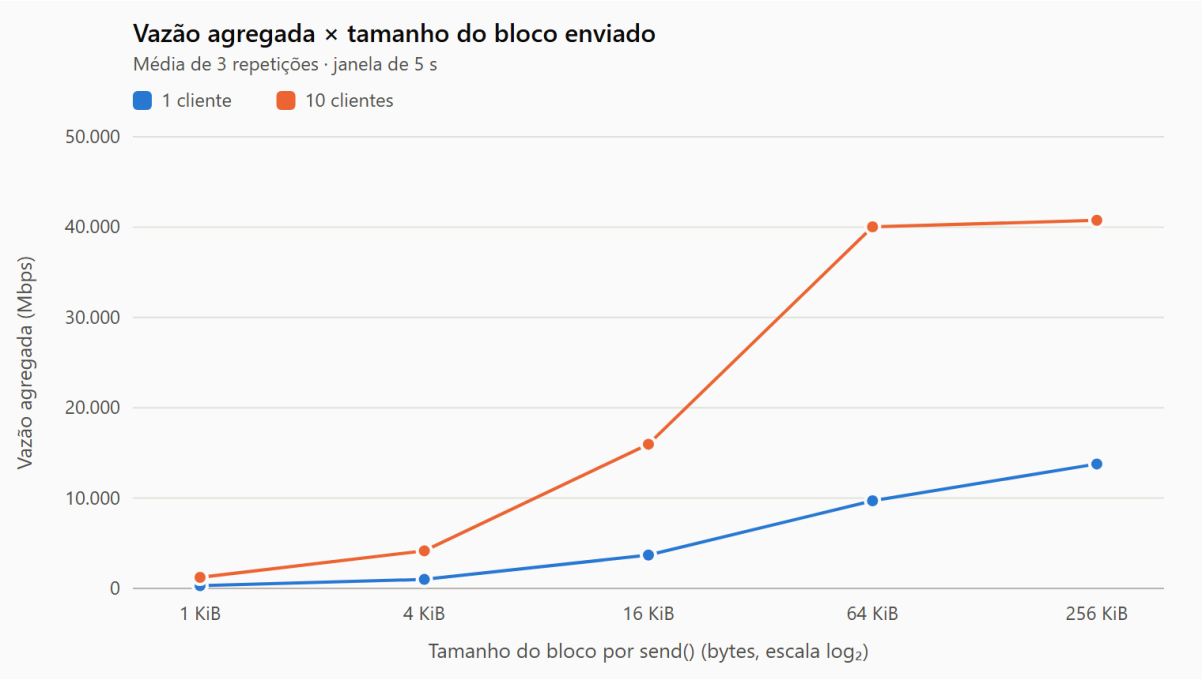
Clientes	Máximo (Mbps)	Média (Mbps)	Desvio-padrão	Média por cliente (Mbps)	CPU do sistema (%)	CPU do servidor (núcleos)	Válidas
1	**12,7**	12,2	0,6	12,2	32,8	0,99	3/3
2	**22,4**	21,3	1,4	10,7	36,2	1,98	3/3
3	**30,9**	30,3	0,6	10,1	46,2	2,96	3/3
4	**40,6**	38,0	2,4	9,5	49,8	3,94	3/3
5	**45,7**	45,6	0,1	9,1	54,6	4,92	3/3
6	**50,4**	48,8	1,5	8,1	64,6	5,86	3/3

Clientes	Máximo (Mbps)	Média (Mbps)	Desvio-padrão	Média por cliente (Mbps)	CPU do sistema (%)	CPU do servidor (núcleos)	Válidas
7	**57,3**	54,6	3,9	7,8	69,6	6,76	3/3
8	**61,9**	60,5	2,0	7,6	77,5	7,62	3/3
9	**66,4**	63,5	2,7	7,1	81,0	8,57	3/3
10	**68,0**	66,2	1,7	6,6	86,7	9,28	3/3



5.3 E3 – Influência do tamanho do bloco

Bloco (bytes)	Clientes	Máximo (Mbps)	Média (Mbps)	Desvio-padrão	CPU do sistema (%)
1.024	1	299,6	293,7	9,8	36,2
4.096	1	1.074,1	1.002,6	104,0	37,4
16.384	1	3.855,9	3.689,2	144,6	36,6
65.536	1	10.014,6	9.706,1	270,2	36,2
262.144	1	14.032,5	13.777,8	368,6	37,3
1.024	10	1.257,2	1.226,3	26,8	96,1
4.096	10	4.249,1	4.158,6	89,9	97,5
16.384	10	16.554,5	15.966,9	807,1	96,5
65.536	10	42.025,9	40.040,0	1.826,5	98,0
262.144	10	41.541,9	40.754,6	1.232,4	99,5



A rodada de 64 KiB do E3 (9,7 Gbps com 1 cliente e 40,0 Gbps com 10) reproduz de forma independente as médias do E1 (9,7 e 40,5 Gbps). Isso indica que o procedimento é repetível.

6. Discussão: impacto do número de clientes

Três regimes aparecem no gráfico de vazão agregada média:

1. **1 → 2 clientes: escala quase linear** (×2,18 na média). Com 1 cliente, um único fluxo TCP não usa toda a máquina: o caminho de envio de uma conexão é essencialmente sequencial (uma thread do cliente, uma do servidor). Um segundo fluxo ocupa núcleos ociosos. A CPU do sistema praticamente não muda, sinal de que havia folga.
2. **2 → 6 clientes: crescimento sublinear.** A média vai de 21,1 a 36,9 Gbps (+75% com 3× mais clientes). A vazão por cliente cai de 10,6 para 6,2 Gbps (gráfico de vazão média por cliente) e a CPU do sistema sobe para 79%. Os fluxos passam a disputar núcleos físicos, cache L3 e banda de memória.
3. **6 → 10 clientes: saturação.** A vazão agregada fica estável em 37 a 40,5 Gbps (+9,6% de 6 para 10 clientes) e a CPU do sistema chega a 98%. Novos clientes apenas dividem a mesma capacidade: a vazão por cliente cai para 4,0 Gbps com 10 clientes.

O processo servidor usa apenas 0,21 núcleo com 1 cliente e 1,32 núcleo com 10 clientes, enquanto a máquina inteira chega a ~15,7 das 16 CPUs lógicas. No loopback do Windows, a maior parte do processamento TCP/IP ocorre no contexto de quem envia (os 10 processos cliente), e o servidor faz basicamente uma cópia kernel → usuário por recv(). Como clientes e servidor dividem a mesma CPU, o experimento mede a capacidade do sistema cliente + servidor nesta máquina, e não apenas a do servidor.

O coeficiente de **variação** ficou entre 2% e 15% na maioria dos cenários, mas chegou a 27% com 1 cliente. A causa é a carga de fundo: na rodada mais lenta (N=1, 6,0 Gbps) a CPU do sistema estava em 79%, contra 31% a 46% nas demais rodadas de N=1, ou seja, outro processo disputou a CPU. Por isso o relatório traz o máximo (melhor capacidade observada, como pedido) e a média com IC 95% (desempenho típico). Para $N \geq 6$ os intervalos de confiança se sobrepõem: estatisticamente, a capacidade é a mesma a partir de 6 clientes.

O índice de Jain ficou ≥ 0,991 (1,000 até 5 clientes). O escalonador e o TCP dividem a banda de forma praticamente igual, e nenhum cliente sofre inanição. A pequena queda acima de 6 clientes coincide com a saturação da CPU, quando alguns processos recebem um pouco menos de tempo de processador.

7. Limites observados e gargalos de desempenho

#	Gargalo	Evidência	Efeito
1	**CPU compartilhada** (16 threads / 8 núcleos físicos, com ~28% já ocupados por carga de fundo)	CPU do sistema de 46% → 98%. A vazão estabiliza quando a CPU satura (N ≥ 6).	Limita a capacidade máxima a ≈ 40 Gbps de média e 45,7 Gbps de pico nesta máquina.
2	**Custo por chamada de sistema e tamanho do bloco**	1 cliente: 0,29 Gbps com 1 KiB contra 13,8 Gbps com 256 KiB (47×). O tempo por bloco segue aproximadamente $T(b) \approx 27\ \mu s + b / 2{,}1\ GB/s$.	Com blocos pequenos, o limite é o número de operações por segundo e não a quantidade de bytes. Com blocos de até 16 KiB, mesmo 10 clientes não passam de 16,6 Gbps.
3	**Protocolo de leitura do chat original** (1 byte por recv())	≈ 12 Mbps por cliente, com uma thread do servidor ocupando 0,93–0,99 núcleo por cliente. Isso é ~0,65 µs por chamada recv(). Com 10 clientes: 66 Mbps e 9,3 núcleos.	≈ 600 a 800× mais lento que a leitura em blocos. É o principal gargalo da aplicação de chat como foi escrita.
4	**Cópias de memória**	40 Gbps = 5 GB/s de *payload*, copiado no mínimo duas vezes (usuário → kernel no send, kernel → usuário no recv). Isso dá ≥ 20 GB/s de tráfego de leitura+escrita, frente a ~44,8 GB/s teóricos do único módulo DDR5-5600 (canal único).	Provavelmente contribui para a saturação. Não foi isolado separadamente (exigiria contadores de hardware).
5	**Modelo uma-thread-por-cliente**	Funciona bem até 10 clientes (Jain ≥ 0,99), mas cada conexão custa uma thread com pilha própria e trocas de contexto.	Com centenas ou milhares de clientes, epoll/IOCP ou um *pool* de threads escalaria melhor. Não foi o limite nesta faixa.
6	**Rede física (fora do teste)**	Loopback não tem enlace físico. A placa Wi-Fi tem enlace de 1,2 Gbps.	Numa rede real, o enlace (≤ 1,2 Gbps no Wi-Fi, 1 Gbps em Ethernet comum) seria o gargalo muito antes do servidor, que no modo bulk sustenta ~30–40× mais.

Limitações do estudo: uma única máquina (clientes e servidor disputam a mesma CPU); medição em loopback; carga de fundo de ~28% de CPU, mitigada pela ordem aleatória e por 5 repetições, mas não eliminada; sem TCP_NODELAY nem ajuste manual de buffers (padrões do SO). O código original era para Linux, mas não foi medido lá pois ninguém do grupo possui máquina Linux, logo foi utilizada uma biblioteca para fazer portabilidade a Windows.

8. Compilação e execução

Instruções completas: README.md.

Resumo:

```
git clone https://github.com/calderar0/redes-sockets-tcp.git
cd redes-sockets-tcp
make          # Linux/macOS  (no Windows/MinGW-w64: mingw32-make)
```

Chat original:

```
./bin/chat_server 5000
./bin/chat_client 127.0.0.1 5000
```

Medição manual (3 clientes, aquecimento de 2s, janela de 10s, blocos de 64 KiB):

```
./bin/chat_server 5000 --bench 3 2 10 --bind 127.0.0.1 &
for i in 1 2 3; do ./bin/chat_client 127.0.0.1 5000 --bench 14 65536 c$i & done; wait
```

Reproduzir todos os experimentos e gerar tabelas e gráficos:

```
python scripts/run_experiments.py --tag main_bulk_64k --clients 1-10 --reps 5 --warmup 2 --window 10
--block 65536 --mode bulk
python scripts/run_experiments.py --tag line_mode --clients 1-10 --reps 3 --warmup 2 --window 5 --block
65536 --mode line
for b in 1024 4096 16384 65536 262144; do python scripts/run_experiments.py --tag block_$b --clients
1,10 --reps 3 --warmup 2 --window 5 --block $b; done
python scripts/analyze.py
```

Entre duas máquinas: no servidor, use --bind 0.0.0.0 (ou omita --bind) e libere a porta no firewall. Nos clientes, use o IP do servidor no lugar de 127.0.0.1.